

Relazione Progetto Algoritmi e Strutture Dati

Corso B di:

- Stefano Talliente 922700;
- Zahra Roshanaie 954526;
- Michele Liquindoli 958446.

Esercizio 1

L'esercizio è stato svolto in linguaggio C.

L'esercizio consiste nell'implementazione di una libreria generica con gli algoritmi di ordinamento: *QuickSort* e *Binary-InsertionSort*.

1. Quicksort è un algoritmo di ordinamento ricorsivo Divide et Conquer.

Tale procedura ricorsiva viene comunemente detta *partition*: preso un elemento chiamato "pivot", si pongono gli elementi minori a sinistra rispetto al pivot e gli elementi maggiori a destra.

L'efficienza dell'algoritmo dipende dalla scelta del perno, infatti abbiamo implementato diversi tipi algoritmi: Lomuto, Hoares.

1. Lomuto -> Sceglie come pivot *l'ultimo* elemento dell'array;
2. Hoares -> Sceglie come pivot il *primo* elemento dell'array;
3. Hoares_r -> Sceglie come pivot *l'elemento randomicamente*;

Abbiamo calcolato il tempo di esecuzione di ognuno di questi algoritmi ottenendo la seguente tabella:

	int	float	string
<i>Lomuto</i>	~8.16 s	~8.28 s	~ +10 min
<i>Hoares</i>	~ 8.60 s	~ 8.91 s	~ 13.73 s
<i>Hoares_r</i>	~ 8.39 s	~ 9.02 s	~ 14.01 s

2. Binary-Insertion Sort è una versione dell'algoritmo Insertion Sort in cui facciamo uso della ricerca binaria per trovare la posizione dell'elemento corrente all'interno della sezione ordinata dell'array.

Binary-InsertionSort	int	float	string
<i>100M</i>	~ 2.37 s	~ 2.38 s	~ 2.34 s
<i>200M</i>	~ 9.93 s	~ 9.86 s	~ 9.31 s
<i>20MLN</i>	~ +10 Min	~ +10 Min	~ +10 Min

Confrontando le tabelle vediamo che il Binary-Insertion Sort si comporta meglio del Quicksort con file di dimensioni piccole, mentre per file di dimensioni grandi non riesce a terminare entro i 10 minuti.

Il Quicksort invece in base alla scelta del pivot cambia la sua efficienza, in particolare vediamo che con l'algoritmo di Lomuto, per le stringhe, non termina entro i 10 minuti mentre le versioni di Hoares riescono a terminare nell'ordine di decine di secondi.

Questo è sostenuto dal momento in cui il quicksort ha una complessità di $O(n \log n)$ mentre il Binary-Insertion Sort ha complessità $O(n^2)$;

Esercizio 2

L'esercizio è stato svolto in linguaggio C.

L'esercizio consiste nell'implementazione di una libreria generica della struttura dati SkipList.

Abbiamo utilizzato questa struttura dati, riempita con un dizionario di parole corrette, per controllare se un file di testo avesse delle parole scritte in modo errato (es. "wita" invece di "vita").

La particolarità è la scelta della costante MAX_HEIGHT. La struttura dati utilizza un concetto probabilistico per stabilire il numero di "vie esprese" di ogni nodo ai nodi successivi.

L'algoritmo fornito per il calcolo probabilistico delle altezze "randomLevel()" *riduce*, secondo una distribuzione geometrica, *le probabilità che un nodo abbia "molte" vie esprese*, cioè produce una probabilità del 50% che il nodo abbia 1 puntatore, una probabilità del 25% che ne abbia 2 e così via.

Questo *evita un uso eccessivo di memoria rispetto ai benefici di velocità* che ne otterremmo in cambio.

Sotto una tabella con degli esperimenti sull'efficienza della SkipList al variare della Costante MAX_HEIGHT

MAX_HEIGHT	5	10	20	30
<i>Tempo ricerca</i>	0.1002 s	0.000271 s	0.000327 s	0.000436 s
<i>Tempo Inserimento</i>	346.8621 s/60= 5.78 min	1.6999 s	1.6880 s	2.2489 s

Infatti se mettiamo la costante ad un valore molto alto come 30, la SkipList ha circa la stessa efficienza di 10. Dopo un certo valore della costante, il suo aumento non risulta in un effettivo miglioramento di prestazioni nonostante il consumo elevato di memoria.

Esercizio 4

L'esercizio è stato svolto in linguaggio JAVA.

Per la nostra libreria di rappresentazione dei Grafi generici in java abbiamo scelto di non avere una classe astratta di Grafo, ma di far specificare come parametro se il grafo è orientato o meno.

Se il grafo Non è orientato, aggiungere un arco "u->v" significa anche l'aggiunta dell'arco "v->u", mentre nei grafi Orientati questo non avviene.

Il nostro grafo tiene traccia dei vertici attraverso una HashMap.

Ogni chiave di questa HashMap è un vertice al quale si associa una LinkedList che contiene i vertici adiacenti rappresentati come Archi.

Abbiamo deciso di non utilizzare una classe Vertex, come un wrapper, per rappresentare il un nodo del grafo.